

System Documentation

Smart Campus Safety & Risk Grid (SCS-RG)

Team Clover

Sadman Islam • Shah Samin Yasar • Ahmed Thousif Thisham

Metropolitan University

GitHub Repo: https://github.com/amisadman/RoboFusion_Techathon_TeamClover

Web Dashboard: <https://clover-scs-rg.vercel.app/>

Backend API Base: <https://robofusion-techathon-teamclover.onrender.com>

IoT Lab Wokwi: <https://wokwi.com/projects/470514529070990337>

Server Room Wokwi: <https://wokwi.com/projects/470509081717871617>

Data Science Lab Wokwi: <https://wokwi.com/projects/470523315735022593>

***Abstract:** This document serves as the complete **System Documentation** for the **Smart Campus Safety & Risk Grid (SCS-RG)**. Built for multi-hazard IoT emergency monitoring and campus safety management, SCS-RG integrates ESP32 hardware sensor nodes, a bounded multi-hazard risk fusion engine, real-time WebSocket state streaming, atomic incident priority dispatching, a predictive logistic regression model, and a natural-language incident parsing engine. This document details per-zone hardware circuit diagrams, software architecture specifications, database ER diagrams, mathematically justified risk fusion formulas, and complete REST API endpoint specifications with example payloads.*

Table of Contents

1. System Overview & Software Architecture

- 1.1 Architectural Layering
- 1.2 Software Architecture Diagram

2. Hardware Sensor Circuit Diagrams (Per Zone)

- 2.1 IoT Lab Node Circuit Diagram
- 2.2 Server Room Node Circuit Diagram
- 2.3 Data Science Lab Node Circuit Diagram

3. Mathematical Risk Calculation Formula & Weight Justifications

- 3.1 Mathematical Fusion Formula
- 3.2 Hazard Weight Justifications
- 3.3 Flame Debounce Filter (N=3 Cycles) & Decay Smoothing
- 3.4 Zone State Classification Thresholds

4. Database Schema & Entity-Relationship Diagram

- 4.1 Database Entity-Relationship (ER) Diagram

5. API Endpoints Reference with Example Requests & Responses

- 5.1 Telemetry Ingestion (POST /api/readings)
- 5.2 Get All Active Zones (GET /api/zones)
- 5.3 Priority Dispatch Queue (GET /api/priority)
- 5.4 Atomic Incident Acknowledgment (POST /api/incidents/:id/ack)
- 5.5 Natural Language Incident Reporting (POST /api/bonus/nl-report)

1. System Overview & Software Architecture

The SCS-RG system is engineered as a decoupled, multi-tiered campus safety grid comprising hardware sensor nodes, an Express/PostgreSQL backend processing engine, and a Next.js 16 command center web interface.

1.1 Architectural Layering

- **Hardware Ingestion Layer:** Distributed ESP32 microcontroller nodes sampling Flame, Gas (MQ2), Water Level, and PIR Occupancy telemetry at 10 Hz.
- **Transport & Security Layer:** Express.js API gateway enforcing `X-Zone-Key` microcontroller authentication and session-based Role-Based Access Control (RBAC).
- **Processing Engine Layer:** Bounded Multi-Hazard Risk Fusion Engine, N=3 Flame Debounce Filter, Exponential Decay Smoother, Atomic Priority Queue Ranker, Logistic Regression Predictor, and Natural Language Parser.
- **Data Persistence Layer:** PostgreSQL relational database managed via Prisma ORM for ACID transactional integrity.
- **Realtime Distribution Layer:** Socket.io WebSocket server broadcasting real-time state changes (`zone:state` , `priority:update` , `incident:opened`) to active operator dashboards.
- **Presentation Layer:** Next.js 16 App Router interface featuring responsive dual-theme command console, live zone grid, dispatch ledger, and Recharts analytics.

1.2 Software Architecture Diagram

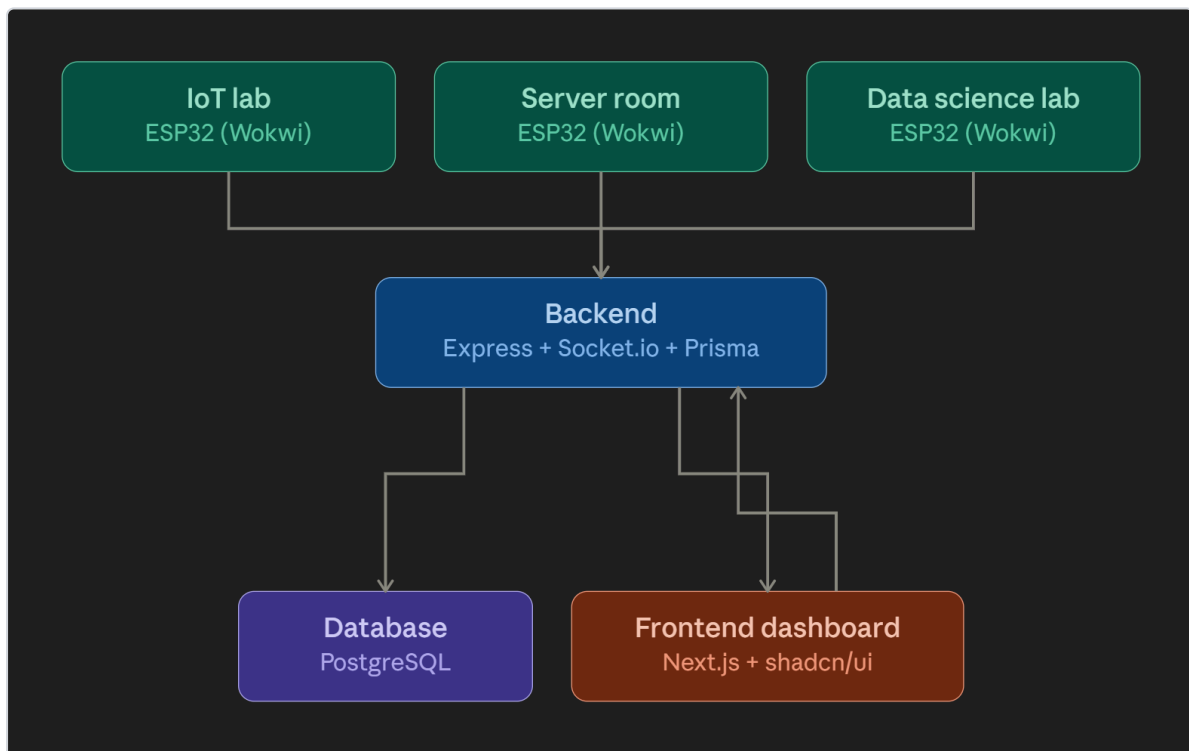


Figure 1: System Component Architecture & High-Level Layering

2. Hardware Sensor Circuit Diagrams (Per Zone)

2.1 IoT Lab Node Circuit Diagram

Monitors soldering stations, chemical storage, and prototype hardware development. Wired with ESP32, IR Flame Sensor (Pin 34), MQ2 Gas Sensor (Pin 35), Water Level Sensor (Pin 32), PIR Motion Sensor (Pin 33), Status LED

(Pin 2), Buzzer (Pin 25), and Emergency Relay Cutoff (Pin 26).

Live Wokwi Simulation: <https://wokwi.com/projects/470514529070990337>

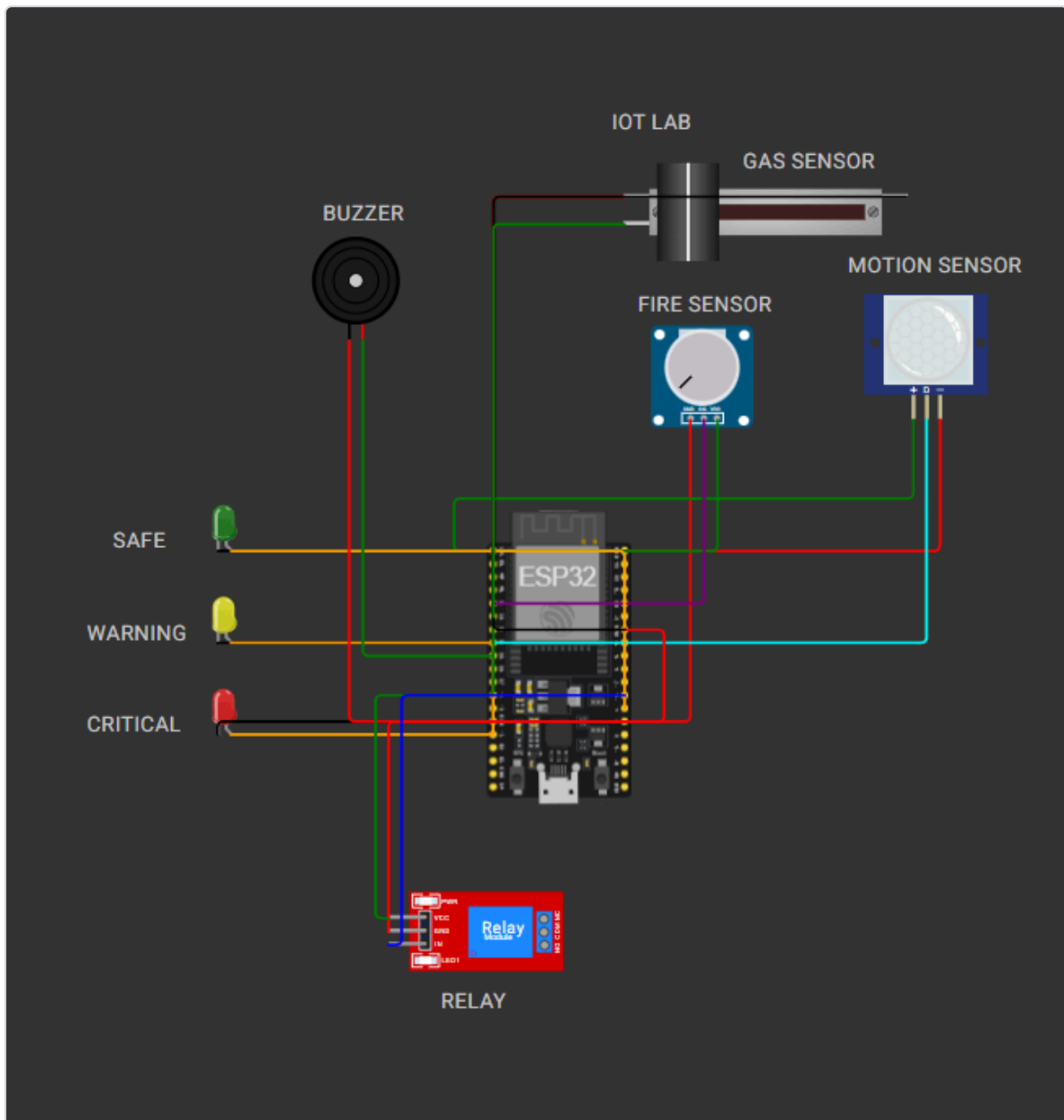


Figure 2: IoT Lab Hardware Sensor Node Circuit Diagram

2.2 Server Room Node Circuit Diagram

Monitors high-density server racks, power distribution units, and electrical switches. Optimized for electrical fire detection and cooling fluid leak monitoring.

Live Wokwi Simulation: <https://wokwi.com/projects/470509081717871617>

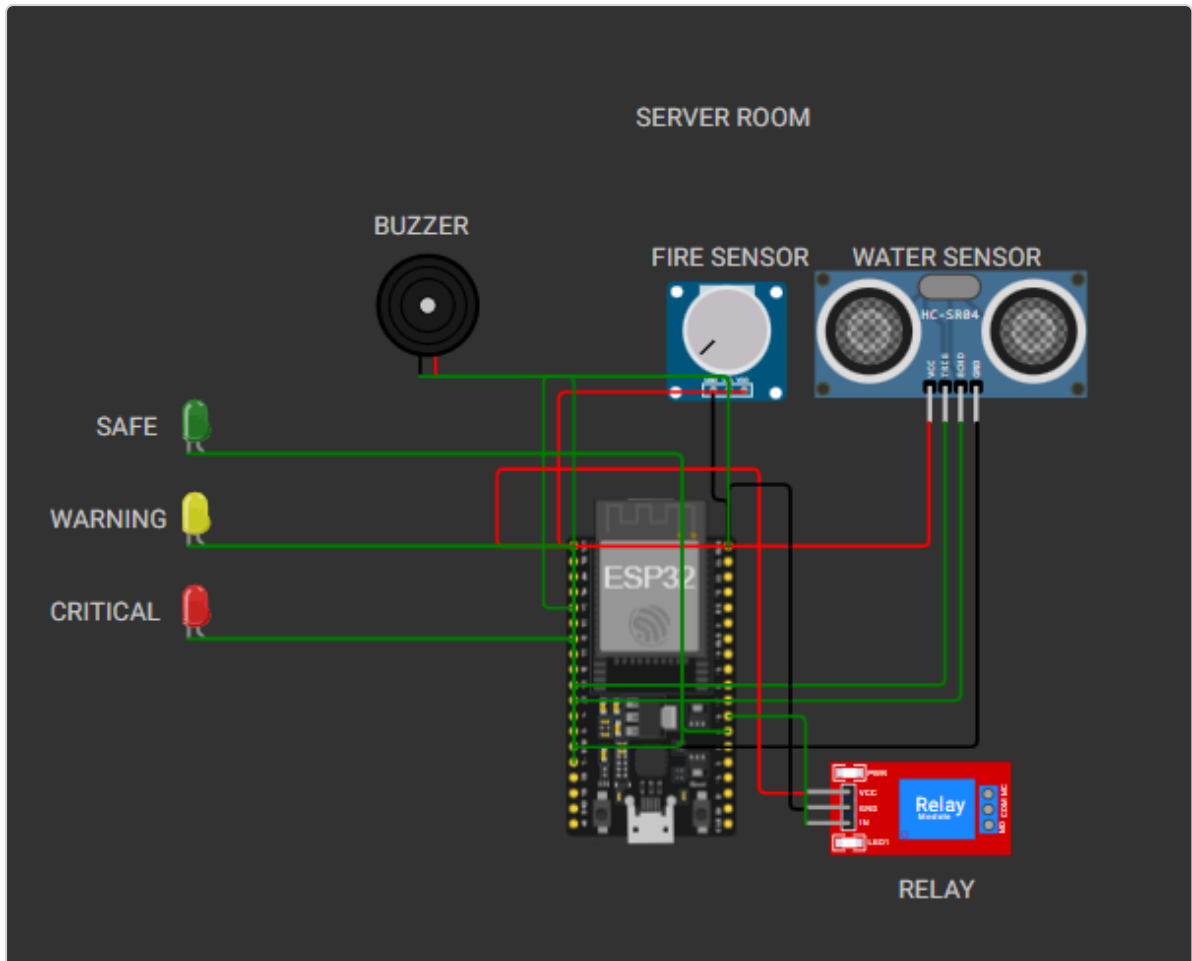


Figure 3: Server Room Hardware Sensor Node Circuit Diagram

2.3 Data Science Lab Node Circuit Diagram

Monitors high-performance computing clusters and student workstation clusters.

Live Wokwi Simulation: <https://wokwi.com/projects/470523315735022593>

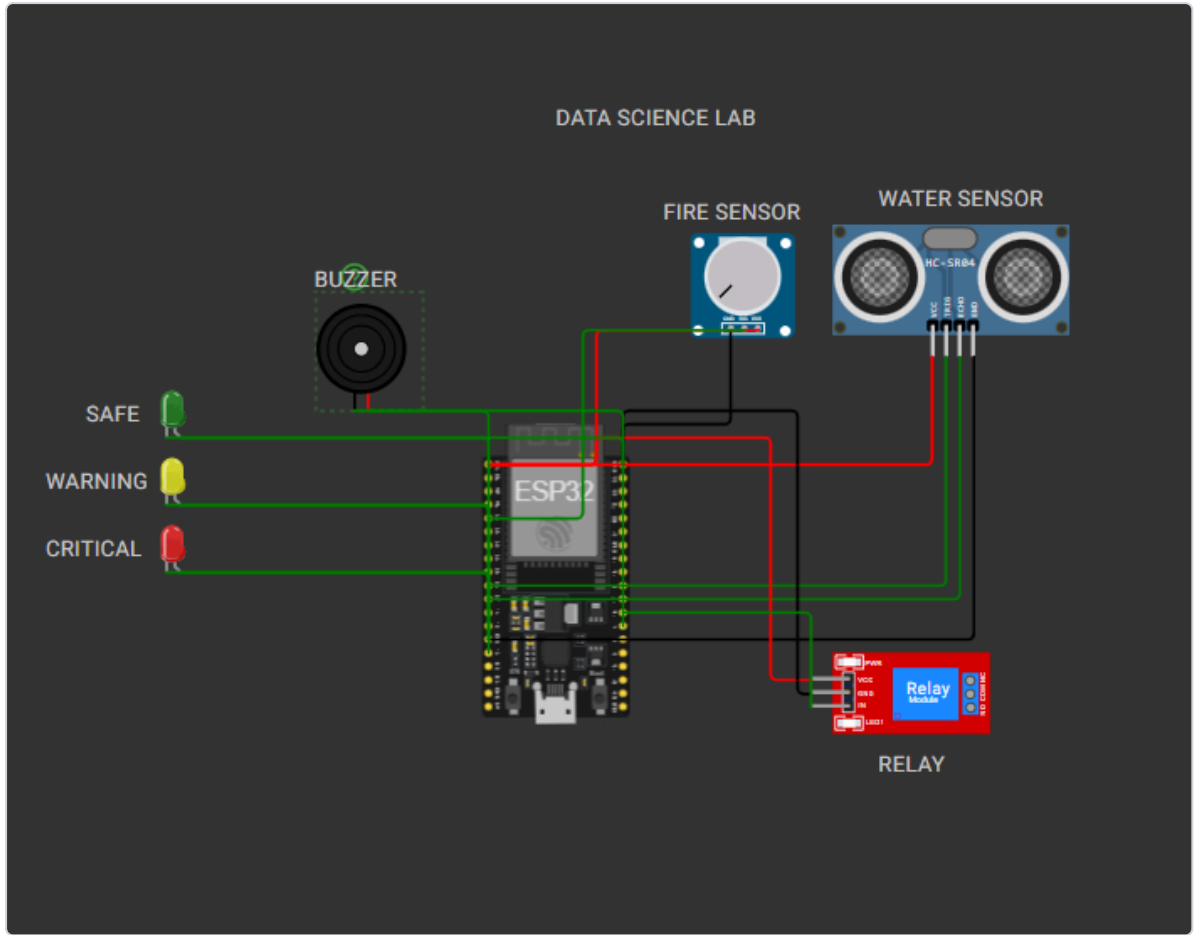


Figure 4: Data Science Lab Hardware Sensor Node Circuit Diagram

3. Mathematical Risk Calculation Formula & Weight Justifications

The core safety engine evaluates overall zone risk using a **Bounded Multi-Hazard Risk Fusion Engine**. It computes a single scalar Risk Score $R \in [0.0, 100.0]$.

$$R = \min(100.0, w_{fire} \times x_{fire} + w_{gas} \times x_{gas} + w_{water} \times x_{water} + w_{occ} \times x_{occ})$$

3.1 Hazard Weight Justifications

Hazard Component	Weight (w_i)	Engineering & Safety Justification
Fire (w_{fire})	65.0	Critical Life Safety Threat: Thermal combustion spreads rapidly and presents immediate catastrophic risk to life and physical assets. Demands top weighting.
Gas (w_{gas})	65.0	Explosion Risk: Combustible gas accumulation (MQ2) creates severe explosion hazards in confined lab spaces. Equal priority to fire.
Water (w_{water})	30.0	Electrical Short & Equipment Submersion: Fluid leaks cause electrical shorts and equipment failure.
Occupancy (w_{occ})	25.0	Human Life Multiplier: Presence of occupants elevates emergency dispatch priority to ensure rapid evacuation.

3.2 Zone State Classification Thresholds

- **SAFE:** Risk Score < 30.0

- **WARNING:** $30.0 \leq \text{Risk Score} < 65.0$
- **CRITICAL:** Risk Score ≥ 65.0
- **OFFLINE:** Sensor health disconnected OR heartbeat silent $> 10s$

4. Database Schema & Entity-Relationship Diagram

The persistence layer uses PostgreSQL managed via Prisma ORM. Below is the complete Entity-Relationship (ER) Diagram:

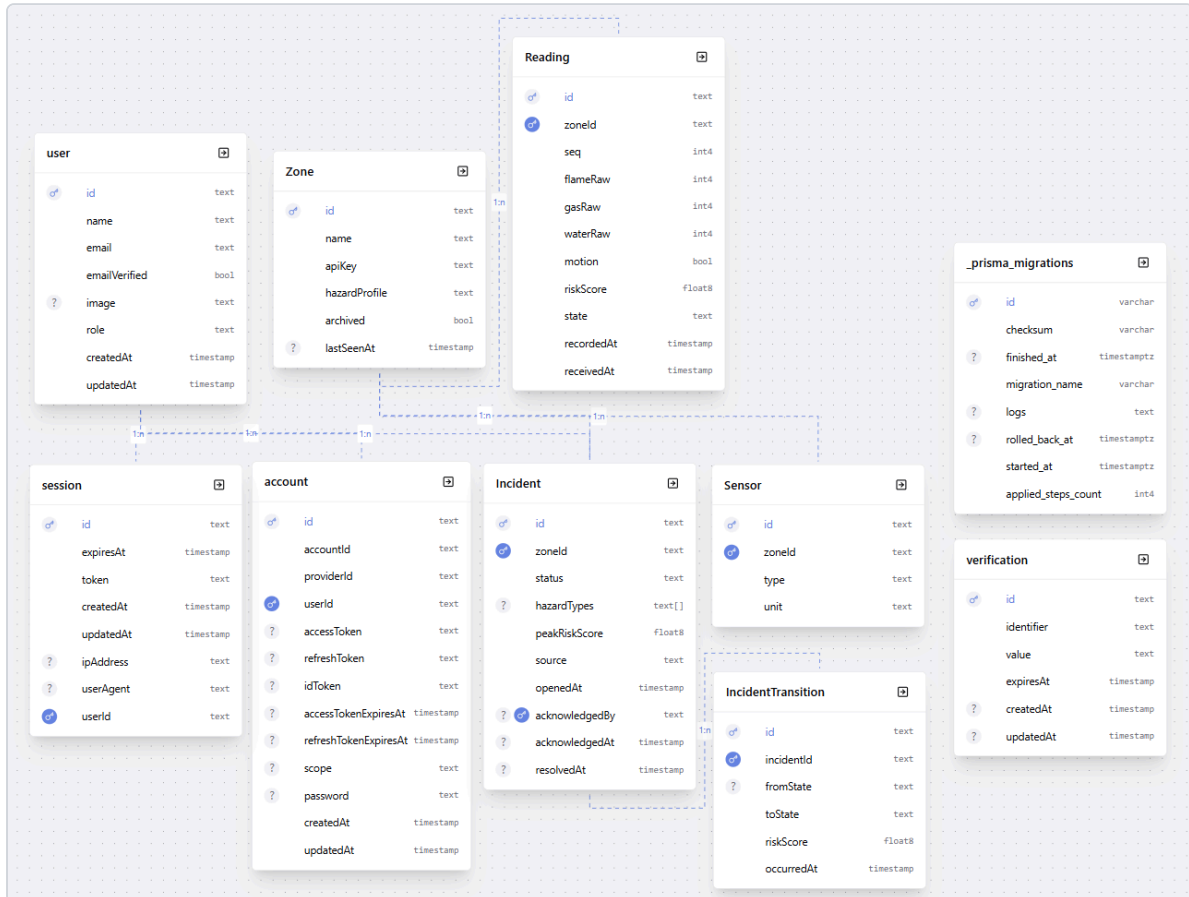


Figure 5: PostgreSQL Database Entity-Relationship (ER) Diagram

5. API Endpoints Reference with Example Requests & Responses

5.1 Telemetry Ingestion (POST /api/readings)

Header: X-Zone-Key: key_server_room_456

```
POST /api/readings HTTP/1.1
Host: robofusion-techathon-teamclover.onrender.com
Content-Type: application/json
X-Zone-Key: key_server_room_456

{
  "zone_id": "server_room",
  "seq": 142,
  "timestamp_ms": 1721985430000,
  "sensors": {
    "flame_raw": 3200,
    "gas_raw": 1800,
```

```

    "water_raw": 150,
    "motion": true
  },
  "sensor_health": { "flame": "ok", "gas": "ok", "water": "ok", "motion": "ok" }
}

```

Response (200 OK):

```

{
  "success": true,
  "message": "Reading ingested successfully",
  "data": {
    "accepted": true,
    "state": "CRITICAL",
    "risk_score": 73.2,
    "commands": { "led": "red", "buzzer": true, "relay_cutoff": true },
    "server_seq_ack": 142
  }
}

```

5.2 Get All Active Zones (GET /api/zones)

Response (200 OK):

```

{
  "success": true,
  "message": "Zones retrieved successfully",
  "data": [
    {
      "zone_id": "server_room",
      "name": "Server Room 101",
      "hazard_profile": "High density electrical & server racks",
      "state": "CRITICAL",
      "risk_score": 73.2,
      "occupied": true,
      "last_seen_at": "2026-07-26T14:30:00.000Z"
    }
  ]
}

```

5.3 Priority Dispatch Queue (GET /api/priority)

Response (200 OK):

```

{
  "success": true,
  "message": "Priority queue retrieved successfully",
  "data": {
    "ranked": [
      {
        "zone_id": "server_room",
        "rank": 1,
        "risk_score": 73.2,
        "occupied": true,
        "seconds_critical": 45,
        "reason": "risk 73.2, occupied, critical 45s",
        "source": "sensor",
        "incident_id": "clx82910a0001"
      }
    ]
  }
}

```



```
]
}
}
```

5.4 Natural Language Incident Reporting (POST /api/bonus/nl-report)

```
{
  "text": "Heavy smoke and electrical fire smell coming from the server room rack 4"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Natural language report processed successfully",
  "data": {
    "parsed": true,
    "extracted_signal": { "zone_id": "server_room", "hazard_type": "fire",
"estimated_severity": "CRITICAL" },
    "validation_gate": "passed",
    "incident_id": "clx99201b0002"
  }
}
```